الدليل الاستراتيجي الشامل للمناقشة التقنية

معمارية الخدمات المصغرة & إدارة الحاويات (Docker & gRPC)

دليل تفصيلي فائق الدقة لشرح مكونات النظام، بروتوكولات التواصل، أنماط المعاملات الموزعة (Saga)، تفادي الـ Deadlock، والرد النموذجي على أسئلة اللجنة.

🚀 التواصل عالي الأداء عبر gRPC

شرح بروتوكول HTTP/2 و Protocol Buffers ولماذا تفوق على REST بنسبة 700% في السرعة.

🐳 هندسة الحاويات (Docker Architecture)

عزل الشبكات Bridge، حماية المنافذ، إدارة الأسرار Secrets، والـ Healthchecks.

🛡️ نمط ساغا (Saga Pattern)

المعاملات التعويضية التلقائية والتعافي عند فشل الشبكة دون الحاجة لـ 2PC.

🔒 منع الـ Deadlock والتزامن

الترتيب الصارم للموارد (Strict Resource Ordering) وقفل الصفوف التشاؤمي InnoDB.

المشروع: نظام التجارة الإلكترونية المعماري الموزع (E-Commerce Platform)

التقنيات: Laravel 11 ▪ Node.js ▪ TypeScript ▪ gRPC ▪ MySQL 8.0 ▪ Docker

المحور الأول: معمارية الخدمات وطريقة التواصل بينها

تم بناء المشروع بالاعتماد على معمارية **الخدمات المصغرة (Microservices Architecture)** مع تطبيق مبدأ **Database-per-Service**، حيث تمتلك كل خدمة قاعدة بيانات معزولة تماماً لضمان الاستقلالية التامة وعدم وجود نقطة فشل مفردة (Single Point of Failure).

الخدمة (Service)	لغة التطوير / الإطار	قاعدة البيانات	المسؤولية الوظيفية الرئيسية
Frontend	React 19 + TypeScript + Vite	N/A (Client Cache)	واجهات المستخدم التفاعلية، إدارة الحالة، واستدعاء الـ APIs.
Backend Gateway	Laravel 11 (PHP 8.2)	MySQL `ecommerce` (Port 3307)	إدارة المنتجات، المستخدمين، التوثيق بـ JWT، وإدارة دورة حياة الطلب.
Inventory Microservice	Node.js 20 + TypeScript	MySQL `inventory_db` (Port 3308)	حجز المخزون اللحظي، فك الحجز، وتفادي البيع الزائد (Overselling).

1.1 لماذا تم اختيار gRPC بدلاً من REST للتواصل الداخلي؟

> 💡 **النقطة الجوهرية التي تبهر لجنة المناقشة:**
>
> التواصل بين `Laravel` و `Inventory Service` يتم عبر **gRPC فوق بروتوكول HTTP/2**
>
> وليس عبر REST/JSON العادي، للأسباب الهندسية التالية:

- **تقنية الـ Multiplexing:** في بروتوكول HTTP/1.1 (المستخدم في REST العادي)، كل استدعاء يحتاج لفتح اتصال TCP جديد أو ينتظر في طابور (Head-of-Line Blocking). بينما في gRPC مع HTTP/2، يتم تمرير مئات الطلبات المتزامنة عبر اتصال TCP وحيد ثنائي الاتجاه (Bi-directional Streaming).
- **ضغط وحجم البيانات (Binary Protocol Buffers):** رسائل JSON ترسل أسماء الحقول كنصوص مكررة (Overhead). بينما Protocol Buffers تحول البيانات إلى بايتات ثنائية مشفرة برقم الحقل فقط، مما يقلل حجم البيانات في الشبكة بنسبة تفوق **80%**.
- **عقود واضحة وصارمة (Strict Interface Contract):** كتابة ملف `.proto` يفرض نمطاً صارماً للأنواع يمنع أخطاء الـ Type Mismatch بين PHP و TypeScript في وقت الـ Compile-Time.

1.2 كود العقد المشترك (Protocol Buffer Definition)

ملف العقد الموحد الذي تم توليد Stubs منه للخدمتين: `proto/inventory.proto`

```
syntax = "proto3";
package inventory;

// تعريف الخدمة والعمليات المتاحة
service InventoryService {
  // 1. فحص المخزون المتاح
  rpc GetStock (GetStockRequest) returns (GetStockResponse);

  // 2. حجز المخزون لعدة منتجات بشكل ذري موحد
  rpc ReserveStock (ReserveStockRequest) returns (ReserveStockResponse);

  // 3. تحرير المخزون المحجوز (Compensating Transaction)
  rpc ReleaseStock (ReleaseStockRequest) returns (ReleaseStockResponse);
}

message ReserveStockRequest {
  repeated ReserveStockItem items = 1;
  string idempotency_key = 2; // مفتاح التحصين ضد التكرار
}

message ReserveStockResponse {
  bool success = 1;
  string reservation_id = 2;
  string failure_reason = 3;
}
```

المحور الثاني: إدارة الحاويات وبيئة Docker بالكامل

تم عزل كافة مكونات النظام داخل بيئة **Docker Compose** متعددة الحاويات (Multi-Container Architecture) لضمان التماثل التام بين بيئة التطوير والإنتاج (Environment Parity).

2.1 الأنماط الأمنية والمعمارية في Docker Compose

1. عزل الشبكات (Network Isolation & Service Discovery)

تم ربط جميع الحاويات عبر شبكة Bridge خاصة تدعى `ecommerce-network`. الحاويات لا تتصل ببعضها عبر أرقام IP المتغيرة، بل باستخدام **DNS الداخلي لـ Docker** (مثلاً: تتصل خدمة لارافيل باسم المضيف `inventory-service:50051`).

🔒 حماية المنافذ الأمنية:

منفذ gRPC الداخلي (`50051`)

غير منشور (Unpublished)

على جهاز المضيف (Host)، مما يمنع أي وصول خارجي إليه؛ بينما التطبيقات الخارجية تتصل فقط بـ Laravel API عبر المنفذ `8000`.

2. إدارة الأسرار الأمنية (Docker Secrets)

بدلاً من حفظ كلمات المرور الحساسة لقواعد البيانات داخل ملف `.env` كنصوص صريحة معرضة للتسريب في Git، تم استخدام ميزة `secrets` في Docker:

```
secrets:
  mysql_password:
    file: ./secrets/mysql_password.txt
  inventory_mysql_password:
    file: ./secrets/inventory_mysql_password.txt
```

يتم تركيب هذه الملفات مؤقتاً في مسار الذاكرة `/run/secrets/` داخل الحاوية وتُقرأ عند الإقلاع فقط.

3. فحوصات الصحة وترتيب الإقلاع (Healthchecks & Startup Ordering)

لتفادي انهيار خدمة المخزون أو لارافيل بسبب محاولة الاتصال بقاعدة البيانات قبل اكتمال إقلاع محرك MySQL، تم تزويد قواعد البيانات بـ `healthcheck`، مع ربط الخدمات التابعة لها بشرط:

```
depends_on:
  mysql:
    condition: service_healthy # لا تبدأ الحاوية إلا بعد استجابة MySQL من ping
```

المحور الثالث: السيناريو الكامل لتدفق البيانات (End-to-End Flow)

ما الذي يحدث عند قيام العميل بالضغط على "تأكيد الطلب" (Checkout) لشراء هاتف وسماعة؟

1 **الواجهة الأمامية (Frontend Request)**

ترسل طلباً بصيغة `POST /api/orders` حاملاً عناصر السلة مع `Idempotency-Key: uuid-v4` ومرفقاً بتوكن المصادقة `Bearer JWT`.

2 **التحقق الأولي في Laravel (OrderService)**

يقوم لارافيل بالتأكد من صلاحية التوكن، ثم يتحقق من أن المنتجات نشطة في جدول `products`، ويجلب الأسعار مباشرة من قاعدة البيانات (وليس من العميل لمنع التلاعب بالسعر).

3 **حفظ حالة العملية (Inventory Operations Table)**

تسجيل سجل في جدول `inventory_operations` بحالة `pending` مع تخزين بصمة الطلب (SHA-256 Hash) لمنع تكرار الإرسال.

4 **استدعاء gRPC الموزع (ReserveStock)**

يقوم عميل `InventoryGrpcClient` بفرز المعرفات تصاعدياً ثم إرسال رسالة ثنائية سريعة إلى `inventory-service:50051`.

5 **القفل الذري وحجز المخزون (Inventory Microservice)**

تفتح الخدمة Transaction في `inventory_db` وتقفل الصفوف المعنية بـ `FOR UPDATE`، تتأكد أن `(total_quantity - reserved_quantity) ⩾ requested`، ثم تقوم بتحديث `reserved_quantity` وإنشاء سجل `reservations` وتأكيد المعاملة (COMMIT).

6 **حفظ الطلب النهائي في قاعدة بيانات التجارة (Order Creation)**

يستلم لارافيل تأكيد الحجز مع `reservation_id`، فيقوم بحفظ الطلب في جدول `orders` وعناصره في `order_items` داخل Transaction محلية، ثم تحويل العملية إلى `completed`.

7 **مسار التعافي التلقائي في حال الفشل (Saga Compensation Fallback)**

إذا فشلت عملية حفظ الطلب في لارافيل لأي سبب بعد نجاح حجز المخزون، يُطلق لارافيل فوراً نداء تعويضي `ReleaseStock(reservation_id)` لفك الحجز من المخزون وإعادة الكمية، مما يضمن اتساق البيانات المطلق!

المحور الرابع: الأنماط الهندسية المتقدمة وحل المعضلات التقنية

4.1 نمط ساغا والمعاملات التعويضية (Saga Pattern)

في المعماريات الموزعة، يستحيل استخدام المعاملات التقليدية ثنائية المراحل (Two-Phase Commit 2PC) بسبب بطئها وتعقيدها وعدم دعمها عبر لغات مختلفة. البديل الصناعي المعتمد هو **Saga Pattern (Orchestration-based)**:

```
// من كود OrderService.php الفعلي في المشروع
try {
    // الخطوة 1: الحجز الموزع في خدمة المخزون
    $reservation = $this->inventoryClient->reserveStock($items, $idempotencyKey);

    // الخطوة 2: حفظ الطلب محلياً
    $order = DB::transaction(function () use (...) {
        // إنشاء الطلب وحفظ العناصر
    });
} catch (\Throwable $e) {
    // إذا تعثر حفظ الطلب بعد حجز المخزون: تشغيل المعاملة التعويضية
    if ($reservationId) {
        try {
            $this->inventoryClient->releaseStock($reservationId);
        } catch (\Throwable $releaseError) {
            // في حال تعطل الشبكة أثناء التعويض، نسجل الفشل في جدول دائم
            FailedCompensation::create([
                'reservation_id' => $reservationId,
                'error' => $releaseError->getMessage(),
            ]);
        }
    }
    throw $e;
}
```

4.2 منع القفل الميت (Deadlock Prevention & Concurrency Control)

> ⚠️ **سؤال دقيق جداً تطرحه اللجان:**
>
> كيف تضمن عدم حدوث Deadlock عند تنافس طلبين في نفس الجزء من الثانية على شراء نفس المنتجات؟

السبب الجذري لـ Deadlock: عندما يحجز الطلب A المنتج 1 ثم يطلب قفل المنتج 2، بينما يحجز الطلب B المنتج 2 ويطلب قفل المنتج 1، ينشأ قفل دائري (Circular Wait) ويقوم MySQL بقتل إحدى المعاملتين بخطأ `ER_LOCK_DEADLOCK`.

الحل الهندسي المنفذ في المشروع: فرض **Strict Resource Ordering (الترتيب الصارم للموارد)**. نقوم بفرز المنتجات تصاعدياً حسب رقم المعرف `product_id` قبل تنفيذ أي قفل في كلٍ من لارافيل وخدمة المخزون:

```
//  من كود inventory-service.ts الفعلي
 // 1. فرز المفاتيح رقمياً لضمان الترتيب الصارم
 const sortedProductIds = [...itemMap.keys()].sort((a, b) => (a < b ? -1 : a > b ? 1 : 0));

 // 2. حجز الأقفال دائماً بنفس الترتيب عبر SELECT ... FOR UPDATE
 for (const productId of sortedProductIds) {
   const [rows] = await connection.query(
     'SELECT product_id, total_quantity, reserved_quantity, ' +
     '(total_quantity - reserved_quantity) AS available_quantity ' +
     'FROM inventory_items WHERE product_id = ? FOR UPDATE',
     [productId]
   );
 }
```

المحور الخامس: بنك الأسئلة المتوقعة من لجنة المناقشة مع الإجابات النموذجية

سؤال 1 **ما الفرق بين معمارية الـ Monolith ومعمارية الـ Microservices في هذا المشروع، ولماذا فصلت المخزون؟**

الإجابة النموذجية:

المونوليث يجمع كل الوظائف وقاعدة البيانات في بيئة واحدة، مما يجعله هشاً في حالات التزامن العالي. قمنا بفصل خدمة المخزون بالذات كـ Microservice لأنها **Hot-spot عالية الأحمال (High-throughput & High-concurrency)** تحتاج لمعالجة عمليات قفل الصفوف اللحظية دون التأثير على تصفح المستخدمين للكتالوج أو المصادقة، كما يمكن توسعتها أفقياً (Scale horizontally) بشكل مستقل تماماً.

سؤال 2 **كيف تحافظ على اتساق البيانات (Data Consistency) بدون وجود قاعدة بيانات موحدة (ACID)؟**

الإجابة النموذجية:

نعتمد على مبدأ **Eventual Consistency (الاتساق المرحلي)** من خلال تطبيق **Saga Pattern**. في حال فشل أي مرحلة لاحقة في دورة حياة الطلب، يتم تشغيل معاملة تعويضية (Compensating Transaction) تقوم بفك حجز المخزون تلقائياً عبر gRPC `ReleaseStock`، مع وجود مجدول مهام دوري (Scheduled Background Worker) يعيد معالجة أي تعويضات تعثرت بسبب انقطاع الشبكة.

سؤال 3 **ما هو الـ Idempotency، وكيف قمت بتطبيقه في هذا النظام؟**

الإجابة النموذجية:

الـ Idempotency يعني أن استدعاء نفس العملية عدة مرات بنفس المعطيات يؤدي إلى نفس النتيجة دون أي أثر جانبي مضاعف. قمنا بإلزام طلبات الشراء بإرسال `Idempotency-Key` فريد من نوع UUID مع حساب `SHA-256 Hash` لمحتوى السلة. إذا أعاد المستخدم أو المتصفح إرسال الطلب، يتعرف النظام على المفتاح ويرجع نفس الرد السابق دون خصم المخزون مرتين.

سؤال 4 **لماذا اخترت Node.js لخدمة المخزون بدلاً من كتابتها بـ PHP مع لارافيل؟**

الإجابة النموذجية:

Node.js مبني على محرك **V8 Event-driven Non-blocking I/O**، وهو فائق الكفاءة في معالجة طلبات الشبكة التزامنية المتزامنة (Concurrent I/O Requests). كما أن دعم مكتبات `@grpc/grpc-js` في نظام Node.js الأصلي مدمج وسريع وخفيف ولا يتطلب تثبيت إضافات C على خادم التشغيل مقارنة بـ PHP PECL.

سؤال 5 **كيف تم تطبيق مبدأ الأمان (Security) داخل بيئة الحاويات Docker؟**

الإجابة النموذجية:

1) عزل شبكي كامل: لا يمكن لأي عميل خارجي استدعاء قواعد البيانات أو خدمة gRPC مباشرة. 2) استخدام Docker Secrets لحماية بيانات الاعتماد من التسريب. 3) توثيق جميع استدعاءات API بواسطة JWT (JSON Web Tokens) مشفرة وموقعة. 4) حماية واجهات الـ Backend عبر Rate Limiting لمنع هجمات الـ Brute Force.

🌟 نصائح ذهبية عند التحدث أمام اللجنة:

- **استخدم المصطلحات الهندسية الدقيقة:** ركز على ذكر كلمات مثل: *"Saga Pattern"، "Pessimistic Locking (FOR UPDATE)"، "Idempotency"، "Multiplexing in HTTP/2"، "Exponential Backoff"*.
- **أظهر الثقة في فصل المسؤوليات:** عندما يسألونك عن سبب عدم استخدام حل بسيط كقاعدة بيانات واحدة، وضح أن هذا النظام مصمم لبيئات الإنتاج والمؤسسات الكبرى (Enterprise Grade) التي تستهدف التوسع غير المحدود (Scalability).